NUMPY

Homogeneous vs Heterogeneous elements

Homogeneous = Same (A collection containing the same type of elements is called homogeneous.)
Ex:a=[10,20,30,40,50],a=[1.0,2.0,3.0,4.0,5.0]

Heterogeneous
Hetero = Different(A collection containing different types of elements is called heterogeneous)
Ex: a=[10, "Rafiq", 25.5]

Homogeneous	Heterogeneous
Same type of elements	Different types of elements
[10, 20, 30]	[10, "Rafiq", 30.5]
int, int, int	int, string, float
Same	Different
Ex:Array	Ex:List

Python Array

An array is a **data type** used to store multiple values using a single variable name.
An array contains an ordered collection of data elements where each element is of the same data type and can be referenced by its index position
Syntax for Array: **array(typecode, elements)**
Ex:

```
>> from array import array              # bring the array from Python's array module.
>> a = array('i', [10, 20, 30, 40, 50])  # 'i' specifies that the array contains integers.
>> a = array('f', [10.5, 20.5, 30.5])    # 'f' specifies that the array contains float.
>>a
```

@@

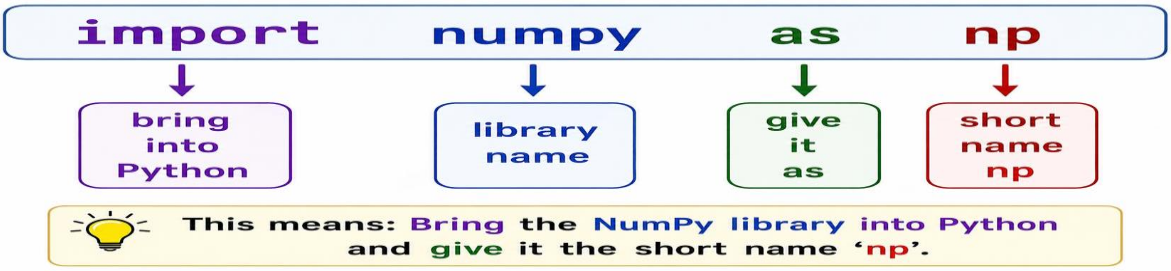
NUMPY

NumPy stands for '**Numerical Python'**.
NumPy is a Python library used for numerical and mathematical operations on arrays.
Note:A library is a collection of **pre-written code** that we can use in our programs
NumPy was introduced by **Travis Oliphant in 2005**.
It is a package for data analysis and scientific computing with Python. NumPy uses a multidimensional array object, and has functions and tools for working with these arrays.

Installing NumPy

NumPy can be installed by typing following command: **pip install numpy**
PIP: Package Installer for Python,

import numpy as np


import
numpy
as
np
bring into Python
library name
give it as
short name np
This means: Bring the NumPy library into Python and give it the short name 'np'.

NumPy libraryని Pythonలోకి తీసుకొని, దానిని np అనే పేరుతో ఉపయోగించు.

np.array() is a **NumPy function** used to create an array.
@@

Numpy Array(ndarray) : NumPy arrays are used to store lists of numerical data, vectors and matrices etc…. **The NumPy array is officially called ndarray**

Numerical means numbers Ex:10,20,30

1D-Array:-A vector is a one-dimensional (1-D) NumPy array that contains a sequence of elements arranged in one direction / one line

Syntax: array_name = np.array([element1, element2, element3, ...])

Ex1:a = np.array([10, 20, 30, 40, 50])

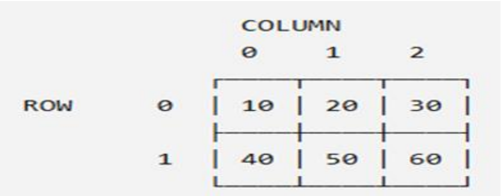
2D-Array: In **Numpy**, a **matrix** is commonly represented using a **2-D Numpy array**.

A **2-D array** contains elements arranged in **rows and columns**.

For a 2-D array, we use: **a [row, column]**

Syntax: array_name = np.array([[element1, element2], [element3, element4]])

Ex2:a= np.array([[10, 20, 30], [40, 50, 60]])



COLUMN

		0	1	2
ROW	0	10	20	30
	1	40	50	60

There are:2 rows, 3 columns,Therefore, this is a 2-D array.

3D Array:A 3-D array is a NumPy array that contains data arranged in three dimensions: tables, rows, and columns.

Syntax: array_name = np.array([2-D array, 2-D array, ...])

Ex3: a3 = np.array([[[10,20,30],[40,50,60]]])

ND Array: An ndarray is a NumPy array that can have one or more dimensions, such as 1-D, 2-D, 3-D, etc.

Syntax: array_name = np.array(data)

Empty array: **a = np.array([])**

1-D → One row
2-D → One table
3-D → Multiple tables

@@

Difference Between List and Array

LIST	ARRAY
List is a Python's built-in collection	ndarray used mainly for numerical data
A Python list can contain **different types** of data Ex:L=[10, "Rafiq", 30.5,'a']	A NumPy array normally contains elements of the **same** data type. Ex:A = np.array([1, 2.5, 4.7])
List Elements are **not** stored contiguously in memory Ex:1000 → 10, 2500 → 20, 4100 → 30, 7300 → 40	Array elements are stored in contiguous memory locations. Ex: 1000 1001 1002 1003 ↓ ↓ ↓ ↓ 10 20 30 40
Lists do **not** support element wise operations L = [10, 20, 30] L * 3 [10, 20, 30, 10, 20, 30, 10, 20, 30]	Arrays support element wise operations A = np.array([10, 20, 30]) A*3 [30 60 90]
lists take **more** space in memory and are less efficient	NumPy array takes up **less** space in memory
List is a part of core Python.	Array (ndarray) is a part of NumPy library.

Creation of NumPy Arrays from List: There are several ways to create arrays.

Step 1:To create an array & to use its methods, first we need to **import** the NumPy library.

```
>>> import numpy as np
```

Step 2: NumPy's **array()** function converts a given list into an array.

```
>>> a = np.array([10,20,30])
```

Step 3: Creating a **1-D** Array, An array with only single row of elements is called 1-D array

```
>>>a = np.array([5,-7.4,'a',7.2])
>>>a
array(['5', '-7.4', 'a', '7.2'], dtype='<U32')
```

Note: Observe that since there is a string value in the list, all integer and float values have been promoted to string, while converting the list to array. **U32** means Unicode-32 data type

Step 4: Creating a **2-D** Array,We can create a two dimensional (2-D) arrays by passing nested lists to the array() function.

```
>>> a = np.array([[2.4,3], [4.91,7],[0,-1]])
array([[ 2.4 ,  3.  ],
      [ 4.91,  7.  ],
      [ 0.  , -1.  ]])
```

Note: Observe that the integers 3, 7, 0 and -1 have been promoted to floats.

* * *

Attributes of NumPy Array: Array Attributes tell us information about an array.

There are **5** important Attributes : **ndim ,shape ,size, dtype ,itemsize**

(A)ndim:Number of Dimensions: ndim tells us how many dimensions the array has.

```
Ex:>>>a=np.array([10,20,30,40,50])
         a.ndim →1
    >>>a=np.array([[10,20,30],[40,50,60]])
    a.ndim →2
```

The row-axis is called axis-0 ,and the column-axis is called axis-1

(B)shape → Rows and Columns,

For a **1-D** array, shape tells us how many elements are present.

```
>>>b=np.array([10,20,30,40,50])
>>>b.shape
```

(5,) The comma in (5,) shows that 5 is the single item in the shape tuple.

For a **2-D** array, shape tells us the number of rows and columns.

```
a=np.array([[10,20,30],[40,50,60]])
a.shape
(2, 3)
```

(C) size → Total Number of Elements: size tells us the total number of elements.

```
Ex:a=np.array([10,20,30,40,50])
     a
     array([10, 20, 30, 40, 50])
     a.size
      5
   Ex:   b=np.array([[10,20,30],[40,50,60]])
         b
         array([[10, 20, 30],
                [40, 50, 60]])
         b.size
         6
```

(d) dtype → Data Type of Elements: dtype tells us the data type of the elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
b = np.array([10.5, 20.5, 30.5, 40.5, 50.5])
c = np.array(["Rafiq", "Ali", "Sara"])
print(a.dtype)
print(b.dtype)
print(c.dtype)
```

(e) Itemsize→ itemsize tells us how many bytes are used to store one element of a NumPy array.

Ex: a.itemsize →8

Other(4) Ways of Creating NumPy Arrays:

Step 1: We can specify **data type** (integer, float, etc.) while creating array using dtype as an argument to array().

Ex:
```
>>>a = np.array( [ [1,2], [3,4] ],dtype=float)
>>>a
array([[1., 2.],
       [3., 4.]])
```

Step 2: Create array of **zeros**→np.zeros(),We can create an array with all elements initialised to 0 using the function zeros().By default, the data type of the array created by zeros() is float.

Ex:
```
>>>a = np.zeros((3,4))
```

Step 3:Create array of **ones**→np.ones(), We can create an array with all elements initialised to 1 using the function ones().By default, the data type of the array created by ones() is float.

Ex:
```
>>>a = np.ones((3,2))
```

Step 4: We can create an array with numbers in a given range and sequence using the **arange()** function. This function is analogous(similar) to the **range()** function of Python.

Ex: a = np.arange(6), a = np.arange(-2, 24, 4)

Indexing in Numpy: For 2-D arrays indexing for both dimensions starts from **0**, and each element is referenced through two indexes i and j, where **i** represents the **row** number & **j** represents the **column** number.

```
>>>a=np.array([["Ramesh", 78, 67, 56],["Vedika", 76, 75, 47],["Harun", 84, 59, 60], ["Prasad", 67, 72, 54]])
>>> a[0,2]
>>> a[1,3]
>>> a[0,4]
```

Slicing in Numpy: Sometimes we need to extract part of an array. This is done through slicing. [start : end]

```
>>>a=np.array([-2,  2,  6, 10, 14, 18, 22])
>>>a
array([-2,  2,  6, 10, 14, 18, 22])
>>>a[3:5]
array([10, 14])
>>>a[::-1]
array([22, 18, 14, 10,  6,  2, -2])
```

@@

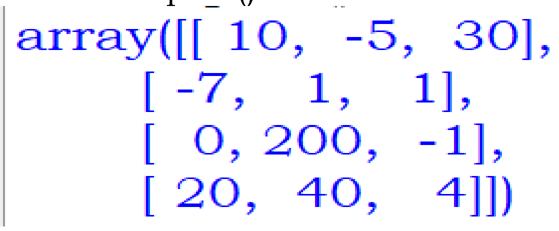
Operations On Arrays: Arithmetic operations on NumPy arrays are fast and simple. When we perform a basic arithmetic operation

```
>>>a1 = np.array([[3,6],[4,2]])
>>>a2 = np.array([[10,20],[15,12]])
>>>a1+a2;a1-a2; a1*a2; a1/a2;a1@a2;a1**3;a2%a1
```

a1+a2	3 + 10 = 13	6 + 20 = 26	4 + 15 = 19	2 + 12 = 14
a1-a2	3 - 10 = -7	6 - 20 = -14	4 - 15 = -11	2 - 12 = -10
a1*a2	3 × 10 =30	6 × 20 = 120	4 × 15 = 60	2 × 12 = 24
a1/a2	3 / 10 = 0.3	6 / 20 = 0.3	4 / 15 = 0.2667	2 / 12 = 0.1667
a1@a2	3 × 10 + 6 × 15 = 120	3 × 20 + 6 × 12 = 132	4 × 10 + 2 × 15 = 70	4 × 20 + 2 × 12 = 104
a1**3	3^3 = 27	6^3 = 216	4^3 = 64	2^3 = 8
a1%a2 a2%a1 Element-wise remainder	3 % 10 = 3 10 % 3 = 1	6 % 20 = 6 20 % 6 = 2	4 % 15 = 4 15 % 4 = 3	2 % 12 = 2 12 % 2 = 0

Transpose: Transposing an array turns its rows into columns and columns into rows

```
>>>a = np.array([[10,-7,0, 20], [-5,1,200,40],[30,1,-1,4]])
>>>a.transpose()
```



```
array([[ 10,  -5,  30],
       [ -7,   1,   1],
       [  0, 200,  -1],
       [ 20,  40,   4]])
```

Sorting:Sorting is to arrange the elements of an array in hierarchical order either ascending or descending. By default, numpy does sorting in ascending order.
By default, sorting is done row-wise (i.e., on axis = 1)

```
>>>a = np.array([1,0,2,-3,6,8,4,7])
>>>a.sort()
>>>a → array([-3,  0,  1,  2,  4,  6,  7,  8])
```

Sorting using 2-D array:

```
>>>a = np.array([[10,-7,0, 20],[-5,1,200,40],[30,1,-1,4]])
>>>a.sort()
>>>a
```

ROW	BEFORE SORTING	AFTER SORTING	OUTPUT
ROW1	10,-7,0, 20	-7,0,10,20	array([[-7, 0, 10, 20], [-5, 1, 40, 200], [-1, 1, 4, 30]])
ROW2	-5,1,200,40	-5,1,40,200	
ROW3	30,1,-1,4	-1,1,4,30	

1-D array: all elements are sorted.
2-D array: a.sort() → **each row is sorted separately by default.**

```
>>> a[::-1]  #reverse the order of the rows.
```

Reshaping Arrays: We can modify the shape of an array using the reshape() function.

```
>>>a = np.arange(10,22)
>>>a
>>>array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21])
>>>a.reshape(3,4),a.reshape(4,3),a(6,2),a(12,1)
```

Statistical Operations On Arrays: NumPy provides functions to perform many useful statistical operations on arrays.

```
>>>a1 = np.array([1,0,2,-3,6,8,4,7])
>>>a2 = np.array([[3,6],[4,2]])
>>> a1.min();a2.max();a1.min();a2.min();a1.sum();a2.sum();a1.mean();a2.mean()
```

a1.max()	a2.max()	a1.min()	a2.min()	a1.sum()	a2.sum()	a1.mean()	a2.mean()
8	**6**	**-3**	**2**	**25**	**15**	**3.125**	**3.75**

a1.sum()→.1 + 0 = 1 → 1 + 2 = 3 → 3 + (-3) = 0 → 0 + 6 = 6 → 6 + 8 = 14 → 14 + 4 = 18 → 18 + 7 = 25
a2.sum() →3 + 6 = 9 → 9 + 4 = 13 → 13 + 2 = 15
a1.mean()→Mean = Sum ÷ Number of elements = 25 ÷ 8 = 3.125
a2.mean()→Mean = Sum ÷ Number of elements = 15 ÷ 4 = 3.75